

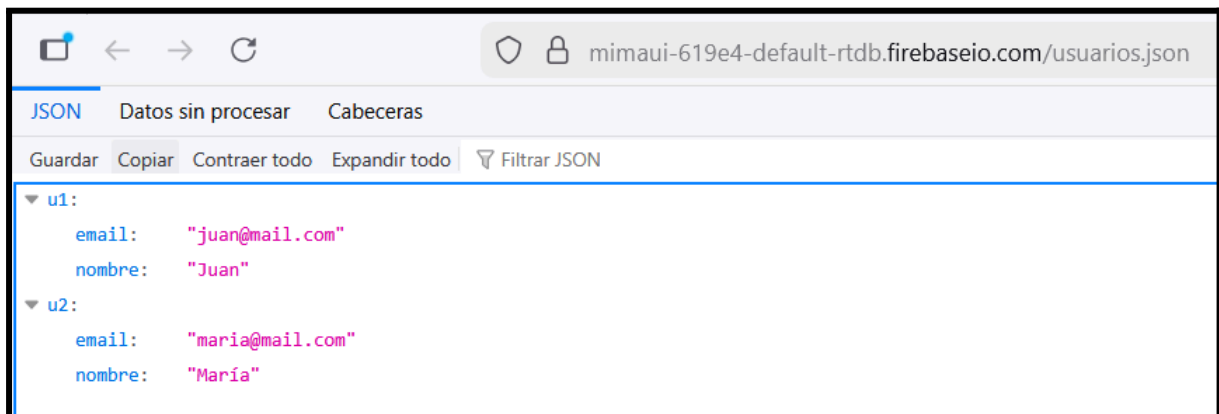
Reto 6 CRUD + Realtime json

Hemos creado la base de datos Realtime en:

<https://docs.google.com/document/d/1Mftk8DP09pD57GjIPWG9WIh-dIMZT0w7nc3rcLUSGDY/edit?tab=t.0>

Verificamos que funciona:

<https://mimaui-619e4-default-rtdb.firebaseio.com/usuarios.json>



Vamos a hacer un CRUD en Angular usando solo HttpClient y la REST API de Firebase Realtime Database.

Creamos un nuevo proyecto en angular: **ng new realtimecrudjson**

En [app.config.ts](#) agregamos **provideHttpClient()**

ng g i interfaces/usuario

```
export interface Usuario {
  id?: string;
  nombre: string;
  email: string;
}
```

ng generate service services/usuarios

En [usuarios.service.ts](#)

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable, map } from 'rxjs';
import { Usuario } from '../interfaces/usuario';
```

```
@Injectable({
  providedIn: 'root',
})
```

```
export class Usuarios {
  private urlBase = 'https://mimaui-619e4-default-rtdb.firebaseio.com/usuarios';

  constructor(private http: HttpClient) { }
```

```

getUsuarios(): Observable<Usuario[]> {
  return this.http.get<{ [key: string]: Usuario }>(this.urlBase + '.json')
    .pipe(
      map(res => {
        if (!res) return [];
        return Object.keys(res).map(key => ({ id: key, ...res[key] }));
      })
    );
}

addUsuario(usuario: Usuario) {
  return this.http.post(this.urlBase + '.json', usuario);
}

updateUsuario(usuario: Usuario) {
  if (!usuario.id) throw new Error('Usuario debe tener id');
  return this.http.put(this.urlBase + '/' + usuario.id + '.json', {
    nombre: usuario.nombre,
    email: usuario.email
  });
}

deleteUsuario(id: string) {
  return this.http.delete(this.urlBase + '/' + id + '.json');
}
}

```

Vamos a desglosar esa línea, que es muy importante para entender cómo se trabaja con Firebase Realtime Database en Angular. La línea es:
return Object.keys(res).map(key => ({ id: key, ...res[key] }));

1 Contexto

Cuando haces un `GET` a Firebase Realtime Database, por ejemplo:

ts

 Copiar código

```
https://mimaui-619e4-default-rtdb.firebaseio.com/usuarios.json
```

Firebase devuelve un **objeto** con todos los usuarios, donde **la clave es el ID generado automáticamente** y el valor es el objeto del usuario:

json

 Copiar código

```
{
  "-N0abc123": { "nombre": "Juan", "email": "juan@mail.com" },
  "-N0abc124": { "nombre": "Ana", "email": "ana@mail.com" }
}
```

2 Object.keys(res)

`Object.keys()` devuelve un **array con todas las claves del objeto**:

ts

 Copiar código

```
Object.keys(res); // ["-N0abc123", "-N0abc124"]
```

Esto nos permite **recorrer cada usuario** usando su ID.

3 `.map(key => ...)`

`map` recorre cada clave del array y devuelve un **nuevo array transformado**.

En este caso, por cada `key` (ID del usuario), hacemos:

ts

 Copiar código

```
({ id: key, ...res[key] })
```

- `id: key` → agregamos el ID de Firebase como propiedad `id` del objeto.
- `...res[key]` → usamos **spread operator** para copiar todas las propiedades del usuario (`nombre`, `email`, etc.) dentro del nuevo objeto.

Por ejemplo, para la clave `"-N0abc123"`:

ts

 Copiar código

```
key = "-N0abc123"  
res[key] = { nombre: "Juan", email: "juan@mail.com" }
```

Entonces:

ts

 Copiar código

```
{ id: key, ...res[key] }  
// se convierte en:  
{ id: "-N0abc123", nombre: "Juan", email: "juan@mail.com" }
```

4 Resultado final

Después de mapear todas las claves:

ts

 Copiar código

```
[
  { id: "-N0abc123", nombre: "Juan", email: "juan@mail.com" },
  { id: "-N0abc124", nombre: "Ana", email: "ana@mail.com" }
]
```

Ahora tenemos un **array de usuarios**, cada uno con su **ID incluido**, listo para usar en Angular (por ejemplo en `*ngFor`).

Resumen rápido:

1. `Object.keys(res)` → obtiene todas las claves (IDs de Firebase).
2. `.map(...)` → recorre cada clave y crea un **objeto completo con el ID incluido**.
3. Devuelve un **array de objetos** que Angular puede manejar fácilmente.

En app.ts

```
import { Component, inject, OnInit, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Usuario } from './interfaces/usuario';
import { Usuarios } from './services/usuarios';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, FormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App implements OnInit {
  protected readonly title = signal('realtimecrudjson');

  usuarios: Usuario[] = [];
  nuevoUsuario: Usuario = { nombre: "", email: "" };
  private service=inject(Usuarios)

  ngOnInit() {
    this.cargarUsuarios();
  }

  cargarUsuarios() {
    this.service.getUsuarios().subscribe(data => {
      this.usuarios = data;
    });
  }

  agregar() {
    this.service.addUsuario(this.nuevoUsuario).subscribe(() => {
      this.cargarUsuarios();
      this.nuevoUsuario = { nombre: "", email: "" };
    });
  }

  actualizar(u: Usuario) {
    this.service.updateUsuario(u).subscribe(() => this.cargarUsuarios());
  }

  eliminar(u: Usuario) {
    if (u.id) {
      this.service.deleteUsuario(u.id).subscribe(() => this.cargarUsuarios());
    }
  }
}
```

¿Qué significa `() => this.cargarUsuarios()`?

Es una función anónima equivalente a:

Ts ^

 Copiar

```
function() {  
  this.cargarUsuarios();  
}
```

Es decir:

- `()` → no recibe parámetros
- `=>` → devuelve lo que hay a la derecha
- `this.cargarUsuarios()` → se ejecuta cuando RxJS llame a `next`

¿Por qué no escribir `next: this.cargarUsuarios()`?

Porque eso **la ejecuta inmediatamente**, antes de que llegue la respuesta del observable.

Ejemplo:

Ts ^

 Copiar

```
next: this.cargarUsuarios() // ❌ se ejecuta en el momento del subscribe
```

Sintaxis	¿Qué hace?	¿Sirve?
<code>next: this.cargarUsuarios</code>	Pasa la función sin ejecutarla, pero pierde <code>this</code>	❌
<code>next: this.cargarUsuarios()</code>	Ejecuta la función inmediatamente	❌
<code>next: () => this.cargarUsuarios()</code>	Ejecuta la función en el momento correcto y con el <code>this</code> correcto	✅

En app.html

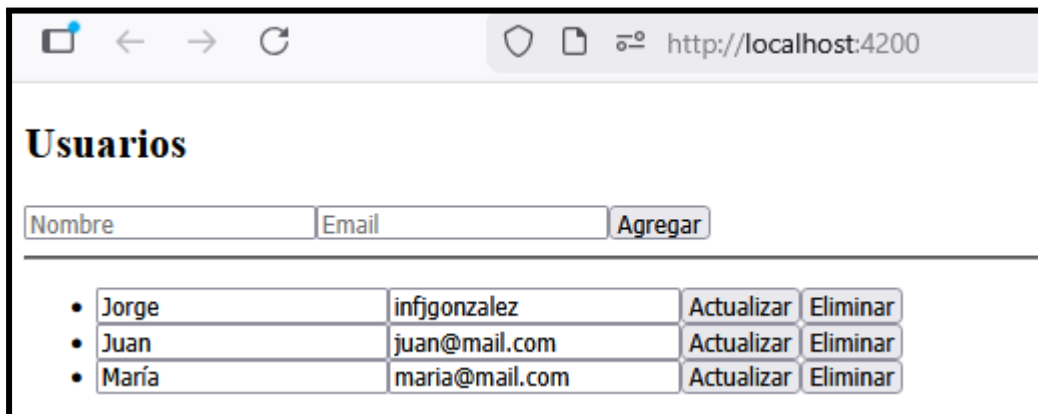
```
<h2>Usuarios</h2>
```

```
<div>
  <input [(ngModel)]="nuevoUsuario.nombre" placeholder="Nombre">
  <input [(ngModel)]="nuevoUsuario.email" placeholder="Email">
  <button (click)="agregar()">Agregar</button>
</div>
```

```
<hr>
```

```
<ul>
  @for (u of usuarios; track u) {
    <li>
      <input [(ngModel)]="u.nombre">
      <input [(ngModel)]="u.email">
      <button (click)="actualizar(u)">Actualizar</button>
      <button (click)="eliminar(u)">Eliminar</button>
    </li>
  }
</ul>
```

RESULTADO



Usuarios			
	Nombre	Email	Agregar
•	Jorge	infjgonzalez	Actualizar Eliminar
•	Juan	juan@mail.com	Actualizar Eliminar
•	María	maria@mail.com	Actualizar Eliminar

En app.ts de otro modo con subscribe, next y error

```
import { Component, inject, OnInit, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Usuario } from './interfaces/usuario';
import { Usuarios } from './services/usuarios';
import { FormsModule } from '@angular/forms';
```

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, FormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App implements OnInit {
  protected readonly title = signal('realtimecrudjson');
```

```
  usuarios: Usuario[] = [];
  nuevoUsuario: Usuario = { nombre: "", email: "" };
  private service = inject(Usuarios)
```

```
  ngOnInit() {
    this.cargarUsuarios();
  }
```

```
  cargarUsuarios() {
    this.service.getUsuarios().subscribe({
      next: (data) => {
        this.usuarios = data;
      },
      error: (err) => {
        console.error('Error al cargar usuarios:', err);
      }
    });
  }
```

```
  agregar() {
    this.service.addUsuario(this.nuevoUsuario).subscribe({
      next: () => {
        this.cargarUsuarios();
        this.nuevoUsuario = { nombre: "", email: "" };
      },
      error: (err) => {
        console.error('Error al agregar usuario:', err);
      }
    });
  }
```

```
  actualizar(u: Usuario) {
    this.service.updateUsuario(u).subscribe({
      next: () => this.cargarUsuarios(),
      error: (err) => {
        console.error('Error al actualizar usuario:', err);
      }
    });
  }
```

```
}  
  
eliminar(u: Usuario) {  
  if (u.id) {  
    this.service.deleteUsuario(u.id).subscribe({  
      next: () => this.cargarUsuarios(),  
      error: (err) => {  
        console.error('Error al eliminar usuario:', err);  
      }  
    });  
  }  
}
```